

Проект для самостоятельного обучения
По теме *“Компьютерное зрение”*
“Защита компьютера от посторонних лиц”

Соколов Иван Дмитриевич
Студент магистратуры 1 курс
Университета им. Лобачевского
Институт ИТММ
Направление ПМИ

г. Нижний Новгород
29 сентября 2025

Оглавление

Цель	2
Описание	2
Функциональная цель	2
Архитектура и технологии	3
Реализация	4
FaceTrainer.py - Модуль обучения системы распознавания лиц	4
FaceDetector.py - Модуль детекции лиц с использованием MediaPipe	5
FaceRecognizer.py - Модуль идентификации "своих/чужих" лиц	5
ComputerGuard.py - Главный координатор системы безопасности.....	7
main.py - Главный запускающий скрипт клиентской части.....	8
TelegramBot.py - Бот для управления системой и получения оповещений.....	10
SupabaseStorage.py - Менеджер базы данных для системы безопасности	12
App.py - FastAPI сервер для системы безопасности	14
Дополнительно	16
Название и создание бота	16
Структура проекта и размещение	16
Клиентская часть (Client).....	16
Виртуальное окружение клиента.....	16
Результаты	17
Документация	20
Литература	22

Цель

Описание

Освоить на практике в рамках самостоятельного обучения компьютерное зрение и серверную разработку. Для этого создадим систему, которая следит через веб-камеру за компьютером и при появлении незнакомца шлет уведомление в Telegram с его фото и скриншотом экрана. В процессе будут изучены OpenCV, FastAPI, Aiogram, Supabase и другие фреймворки.

Функциональная цель

- Система должна:
 - в реальном времени захватывать видео с веб-камеры;
 - детектировать и распознавать лица в кадре.
 - иметь базу "известных" лиц (например, лицо владельца).
- При обнаружении лица, отсутствующего в базе "известных", система должна:
 - сделать и сохранить фотографию незнакомца;
 - сделать и сохранить скриншот рабочего стола;
 - отправить в Telegram-бота сообщение с этими изображениями.
- Пользователь должен иметь возможность управлять системой через Telegram-бота (например, запуск/остановка, добавление новых известных лиц).

Архитектура и технологии

- **Архитектура:**

- **Клиентская часть (на компьютере под наблюдением):** Скрипт на Python (OpenCV, MediaPipe), который работает с камерой.
- **Серверная часть (Backend):** REST API на FastAPI. Отвечает за логику, работу с БД, коммуникацию с ботом.
- **База данных:** Supabase. Хранит данные пользователей и привязку к компьютерам.
- **Telegram Bot:** На Aiogram. Обеспечивает интерфейс для уведомлений и управления.
- **Хостинг:** [Render.com](https://render.com) для развертывания сервера.

- **Стек технологий:**

- **Язык программирования:** Python
- **Фреймворки и библиотеки:**
 - FastAPI (веб-сервер)
 - Aiogram (Telegram бот)
 - OpenCV (компьютерное зрение, работа с видео)
 - MediaPipe (детекция лиц)
- **База данных:** Supabase (PostgreSQL)
- **Хостинг и контроль версий:**
 - GitHub (хранение кода)
 - [Render.com](https://render.com) (деплой сервера)

Реализация

FaceTrainer.py - Модуль обучения системы распознавания лиц

Код реализует класс для обучения модели распознавания лиц на основе датасета известных людей. Модуль подготавливает систему к распознаванию "своих" лиц во время основной работы.

Основные функции:

1. **Инициализация LBPН-распознавателя** - использует алгоритм Local Binary Patterns Histograms из OpenCV
2. **Обработка датасета** - проходит по папкам с именами людей и обрабатывает все изображения (JPG, PNG, JPEG)
3. **Предобработка изображений:**
 - Конвертация в grayscale
 - Ресайз до 100x100 пикселей для стандартизации
4. **Создание меток** - присваивает числовые метки каждому человеку и сохраняет соответствие "метка-имя"
5. **Обучение модели** - на подготовленных данных лиц и меток
6. **Сохранение результатов:**
 - Модель face_model.yml
 - Метки и mapping в labels.json
 - Метаданные обучения в metadata.json

Ключевые особенности:

- Подробное логирование процесса
- Обработка ошибок при загрузке изображений
- Проверка наличия датасета и изображений
- Генерация понятных отчетов о процессе обучения

FaceDetector.py - Модуль детекции лиц с использованием MediaPipe

Код реализует класс для обнаружения лиц на изображениях с помощью технологии MediaPipe от Google.

Основные функции:

1. Инициализация детектора MediaPipe:

- `model_selection=0` - оптимизирован для ближних лиц (рекомендуется для веб-камер)
- `min_detection_confidence=0.5` - порог уверенности для детекции

2. Детекция лиц:

- Конвертирует BGR (OpenCV) в RGB (MediaPipe)
- Обработывает изображение через MediaPipe Face Detection
- Извлекает bounding boxes обнаруженных лиц

3. Преобразование координат:

- Конвертирует относительные координаты MediaPipe (0.0-1.0) в абсолютные пиксельные координаты
- Возвращает bounding boxes в формате (x, y, width, height)

Ключевые особенности:

- Обнаружение **ЛЮБЫХ** лиц (без различения "свой/чужой")
- Высокая производительность благодаря оптимизированному MediaPipe
- Автоматическое освобождение ресурсов при уничтожении объекта
- Обработка ошибок для стабильной работы

Вход/Выход:

- **Вход:** Изображение в формате BGR (стандарт OpenCV)
- **Выход:** Список bounding boxes обнаруженных лиц или пустой список при их отсутствии

Этот модуль служит первой стадией в пайплайне распознавания - находит все лица в кадре, которые затем будут классифицированы как "свои/чужие" другими компонентами системы.

FaceRecognizer.py - Модуль идентификации "своих/чужих" лиц

Код реализует классификатор, который отличает известные лица от незнакомцев с помощью обученной LBPН-модели.

Основные функции:

1. Загрузка обученной модели:

- Загружает файлы `face_model.yml` и `labels.json` из папки `known_faces_db`
- Восстанавливает mapping между метками и именами людей

2. Классификация лиц:

- Принимает обрезанное изображение лица (BGR)
- Конвертирует в grayscale и ресайзит до 100x100 пикселей
- Использует predict() для получения метки и уверенности распознавания

3. Логика определения "чужого":

- **confidence < 70** - лицо распознано как "свой"
- **confidence ≥ 70** - лицо считается "чужим"
- При отсутствии модели все лица считаются незнакомцами

Ключевые особенности:

- **Порог уверенности 70** - баланс между точностью и чувствительностью
- **Обработка ошибок** - при сбоях распознавания лицо считается чужим
- **Логирование результатов** - записывает кто распознан и с какой уверенностью

Работа в пайплайне:

1. FaceDetector находит все лица в кадре
2. FaceRecognizer проверяет каждое лицо на принадлежность к "своим"
3. Возвращает True для незнакомцев, False для известных людей

Это центральный модуль системы безопасности, который принимает решение о необходимости отправки оповещения.

Код реализует основной класс, который управляет всем пайплайном мониторинга и оповещений.

Основные функции:

1. Инициализация системы:

- Генерация/загрузка уникального computer_id
- Загрузка конфигурации API из api_config.json
- Инициализация детектора и распознавателя лиц
- Настройка порогов срабатывания (по умолчанию: 20 обнаружений за 60 сек)

2. Управление детекциями:

- Ведёт счетчик обнаружений в скользящем окне времени
- Сбрасывает флаг оповещения при снижении активности
- Отслеживает временные метки обнаружений

3. Сбор доказательств:

- capture_stranger_photo() - сохраняет фото с веб-камеры
- take_screenshot() - делает скриншот рабочего стола
- Автоматическое именование файлов с timestamp и computer_id

4. Отправка оповещений:

- send_api_alert() - отправляет POST-запрос на сервер с данными и файлами
- Формирует структуру alert_data с метаданными
- Обрабатывает ответ сервера и ошибки сети

5. Главный цикл мониторинга:

- Захват видео с веб-камеры
- Поточная обработка кадров через process_frame()
- Интеграция с FaceDetector и FaceRecognizer
- Логика принятия решения об отправке оповещения

Ключевые особенности:

- **Пороговая логика** - предотвращает спам уведомлениями
- **Комплексные доказательства** - фото нарушителя + скриншот экрана
- **Уникальная идентификация** - computer_id для привязки к Telegram-пользователю
- **Управление состоянием** - отслеживание отправленных оповещений

Код является точкой входа в систему безопасности, которая запускается на защищаемом компьютере.

Основные функции:

1. Инициализация конфигурации:

- Загружает настройки из config.txt через ConfigLoader
- Настраивает уровень логирования (INFO по умолчанию)
- Предусматривает fallback если конфигурация недоступна

2. Предварительные проверки:

- Проверяет существование обученной модели face_model.yml
- Выводит текущие настройки системы для пользователя
- Предупреждает о необходимости обучения если модель отсутствует

3. Запуск системы охраны:

- Импортирует и создает экземпляр ComputerGuard
- Передает конфигурацию в охранную систему
- Запускает основной цикл мониторинга

4. Обработка завершения:

- Корректно обрабатывает Ctrl+C для graceful shutdown
- Вызывает stop_monitoring() при прерывании
- Добавляет паузу перед выходом для видимых терминалов

Ключевые особенности:

- **Structured startup** - четкая последовательность инициализации
- **Error resilience** - обработка отсутствующих компонентов
- **User-friendly interface** - понятные сообщения о статусе системы
- **Graceful shutdown** - корректное завершение по сигналу прерывания

Рабочий процесс:

1. Проверка зависимостей и конфигурации
2. Валидация обученной модели
3. Запуск ComputerGuard с передачей настроек
4. Мониторинг до ручного прерывания
5. Корректное завершение работы

Скрипт объединяет все компоненты в законченную систему, готовую к развертыванию на целевом компьютере.

TelegramBot.py - Бот для управления системой и получения оповещений

Код реализует Telegram-бота на aiogram, который служит интерфейсом между пользователем и системой безопасности.

Основные функции:

1. Инициализация бота:

- Создание бота с HTML-разметкой и подключением к Supabase
- Регистрация всех команд и обработчиков

2. Система команд для пользователя:

- /start - приветствие и список команд
- /register COMPUTER_ID - привязка компьютера к пользователю
- /status - статус привязанной системы
- /stats - общая статистика системы
- /alerts - история уведомлений
- /help - справка по командам
- /wakeur - "пробуждение" сервера (актуально для [Render.com](https://render.com))

3. Логика привязки устройств:

- Проверка уникальности computer_id (один компьютер → один пользователь)
- Регистрация в базе данных через SupabaseStorage
- Валидация формата команды регистрации

4. Отправка оповещений:

- send_alert_to_user() - основной метод для уведомлений
- Поиск пользователя по computer_id в базе
- Отправка текстового сообщения + фото незнакомца + скриншот
- Сохранение уведомления в историю

5. Работа с историей:

- Получение и форматирование последних уведомлений
- Статистика по количеству срабатываний
- Корректное отображение временных меток

Ключевые особенности:

- **Интуитивный интерфейс** - понятные команды и ответы
- **Защита от дублирования** - проверка привязки компьютеров
- **Комплексные уведомления** - текст + визуальные доказательства

- **История событий** - отслеживание всех срабатываний системы
- **Обработка ошибок** - устойчивость к проблемам с файлами и сетью

Взаимодействие с системой:

1. Пользователь привязывает компьютер через /register
2. Система охраны обнаруживает незнакомца
3. Сервер вызывает send_alert_to_user()
4. Бот отправляет уведомление с фото и скриншотом
5. Пользователь может просматривать историю через /alerts

Бот служит центральным узлом для получения уведомлений и управления настройками системы безопасности.

SupabaseStorage.py - Менеджер базы данных для системы безопасности

Код реализует слой работы с базой данных через Supabase (PostgreSQL), обеспечивая хранение и управление данными системы.

Основные функции:

1. Инициализация подключения:

- Загрузка credentials из переменных окружения
- Проверка подключения тестовым запросом
- Обработка ошибок соединения

2. Управление пользователями:

- register_user() - регистрация и привязка computer_id к user_id
- get_user_by_computer_id() - поиск пользователя по ID компьютера
- get_computer_by_user_id() - поиск компьютера по ID пользователя
- get_user_info() - получение полной информации о пользователе

3. Работа с уведомлениями:

- save_alert() - сохранение события обнаружения незнакомца
- get_alerts_by_computer() - история уведомлений для конкретного компьютера
- get_recent_alerts() - последние уведомления по всей системе

4. Административные функции:

- get_all_users() - список всех зарегистрированных пользователей
- get_all_computers() - список всех привязанных компьютеров
- get_stats() - общая статистика системы
- delete_user() - удаление пользователя (для админки)

Ключевые особенности:

- **UPSERT операции** - обновление или создание записей пользователей
- **Проверка уникальности** - компьютер может быть привязан только к одному пользователю
- **Сортировка и лимиты** - эффективная работа с историей уведомлений
- **Обработка API ошибок** - устойчивость к сетевым проблемам
- **Подробное логирование** - отслеживание всех операций с БД

Структура данных:

- **users таблица:** user_id, computer_id, username, first_name, registered_at
- **alerts таблица:** computer_id, detection_count, message, created_at

Интеграция с системой:

- TelegramBot использует для привязки устройств и отправки уведомлений
- ComputerGuard использует для идентификации компьютеров
- Обеспечивает персистентность данных между перезапусками системы

Модуль служит надежным хранилищем состояния всей системы, обеспечивая сохранность данных о пользователях и событиях безопасности.

App.py - FastAPI сервер для системы безопасности

Код реализует основной веб-сервер на FastAPI, который служит центральным узлом для коммуникации между клиентами и Telegram-ботом.

Основные функции:

1. Инициализация приложения:

- Создание FastAPI приложения с CORS middleware
- Загрузка переменных окружения (Telegram токен, Supabase ключи)
- Инициализация Telegram бота при старте

2. Управление Webhook:

- `startup_event()` - автоматическая настройка webhook при запуске
- `set_webhook()` - установка webhook URL для Telegram
- Эндпоинты для ручного управления webhook (`/set-webhook`, `/delete-webhook`)

3. Обработка Telegram обновлений:

- `POST /webhook` - основной endpoint для входящих сообщений от Telegram
- `GET /webhook` - проверка работоспособности webhook

4. API для клиентов:

- `POST /api/alert` - прием уведомлений от клиентских приложений
- Фоновая обработка с `BackgroundTasks`
- Сохранение и автоматическая очистка временных файлов

5. Системные эндпоинты:

- `GET /` - корневой endpoint с информацией о системе
- `GET /health` - health check для мониторинга
- `GET /webhook-info` - информация о текущем webhook

Ключевые особенности:

Webhook Management:

- Автоматическая настройка на [Render.com](https://render.com)
- Ручное управление для отладки
- Проверка статуса webhook

Обработка оповещений:

- Асинхронная обработка через `BackgroundTasks`
- Автоматическая очистка временных файлов
- Интеграция с `TelegramBot` для отправки уведомлений

Отказоустойчивость:

- Подробное логирование всех операций
- Обработка ошибок на всех уровнях
- Graceful degradation при недоступности бота

Для развертывания:

- Настроены переменные окружения для [Render.com](https://render.com)
- CORS для кросс-доменных запросов
- Health checks для мониторинга

Сервер объединяет все компоненты системы в единое целое, обеспечивая надежную доставку уведомлений о безопасности от клиентов к конечным пользователям через Telegram.

Дополнительно

Название и создание бота

BlackCat - Telegram бот системы безопасности

Бот был создан через **@BotFather** в Telegram с следующими характеристиками:

- **Имя:** BlackCat (отражает концепцию "сторожевого кота" для компьютера)
- **Username:** @SSS_blackcat_bot
- **Описание:** Special Security System <<Black Cat>>
- **Команды:** Настроены все основные команды (/start, /register, /status, /alerts, /help, /wakeur)

Структура проекта и размещение

Серверная часть (Backend)

- **Репозиторий:** <https://github.com/SokIvan/BlackCat>
- **Технологии:** FastAPI, Aiogram, Supabase, Uvicorn
- **Хостинг:** Развернуто на [Render.com](https://render.com) с автоматическим деплоем из GitHub
- **Файлы:**
 - app.py - основной сервер
 - telegram_bot.py - логика бота
 - supabase_storage.py - работа с базой данных
 - requirements.txt - зависимости Python

Клиентская часть (Client)

- **Размещение:** Яндекс.Диск - <https://disk.yandex.ru/d/i1jzgcc8I0axnQ>

Виртуальное окружение клиента

Для клиентской части подготовлено готовое виртуальное окружение со всеми зависимостями.

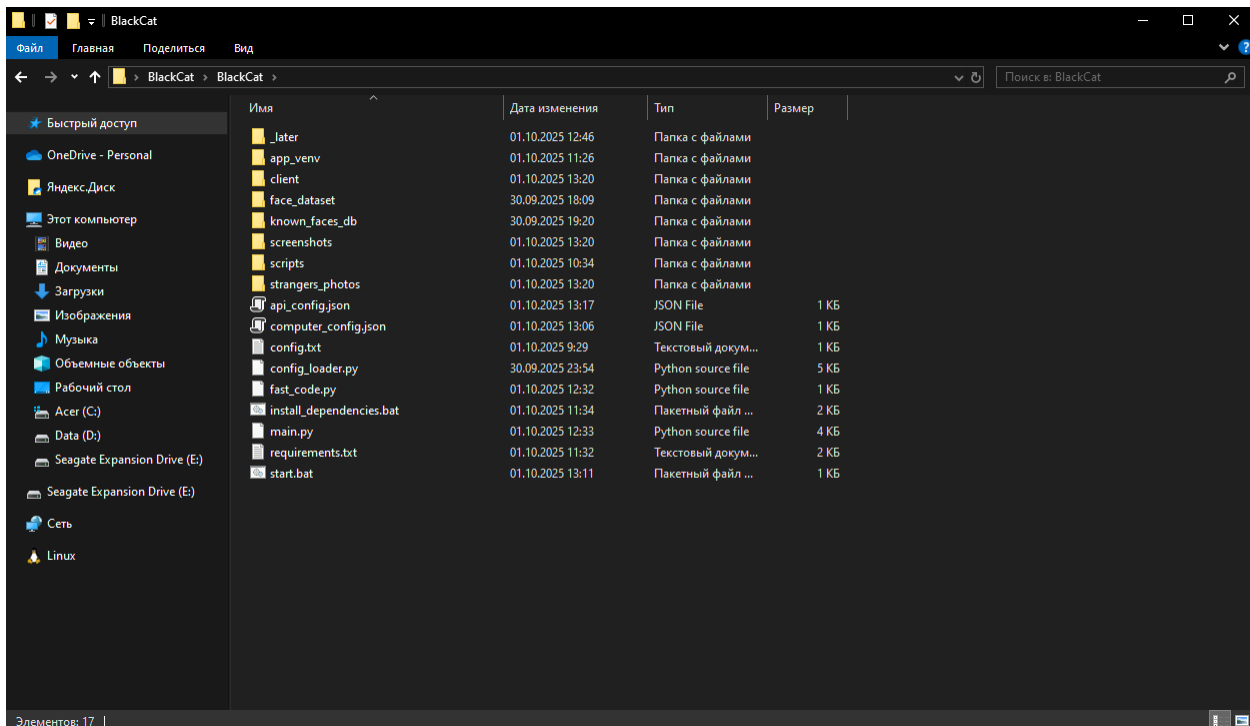
Такая структура позволяет:

- Легко распространять клиентскую часть
- Автоматически обновлять сервер через GitHub
- Обеспечивать надежную работу 24/7 на облачном хостинге
- Минимизировать сложность установки для конечных пользователей

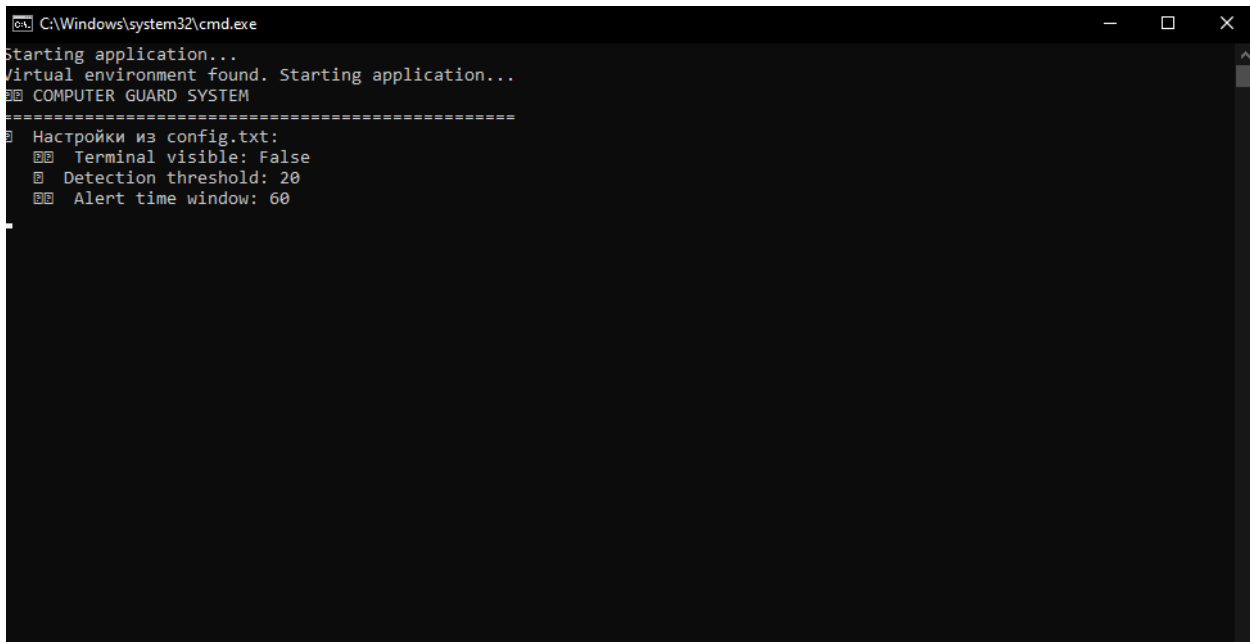
Результаты

Проведем эксперимент, чтобы убедиться, что все работает.

1. После скачивания с Yandex Disk нашего клиента, открываем папку:



2. Для запуска приложения пользователем была создана кнопка start.bat. Нажимаем на нее два раза:



Запускается дополнительное окно. В дальнейшем предполагается запускать скрипт без окна, но для проверки логирования продолжим с окном:

```

C:\Windows\system32\cmd.exe
Starting application...
Virtual environment found. Starting application...
COMPUTER GUARD SYSTEM
=====
Настройки из config.txt:
  Terminal visible: False
  Detection threshold: 20
  Alert time window: 60
2025-10-01 14:49:45,068 - client.computer_guard - INFO - Загружен существующий ID: 1D28A8AD
2025-10-01 14:49:45,068 - client.face_detector - INFO - Детектор лиц (MediaPipe) инициализирован
2025-10-01 14:49:45,303 - client.face_recognizer - INFO - Загружена модель для 2 лиц
2025-10-01 14:49:45,303 - client.computer_guard - INFO - Идентификатор компьютера: 1D28A8AD
2025-10-01 14:49:45,303 - client.computer_guard - INFO - Настройки: threshold=20, window=60s
2025-10-01 14:49:45,303 - client.computer_guard - INFO - Система охраны инициализирована

Запуск системы охраны...
Нажмите Ctrl+C для остановки
=====
2025-10-01 14:49:45,318 - client.computer_guard - INFO - Запуск мониторинга...
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
[0000] 00:00:1759319385.674420 2748 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Disabling support for feedback tensors.
2025-10-01 14:49:47,216 - client.computer_guard - INFO - Камера успешно подключена

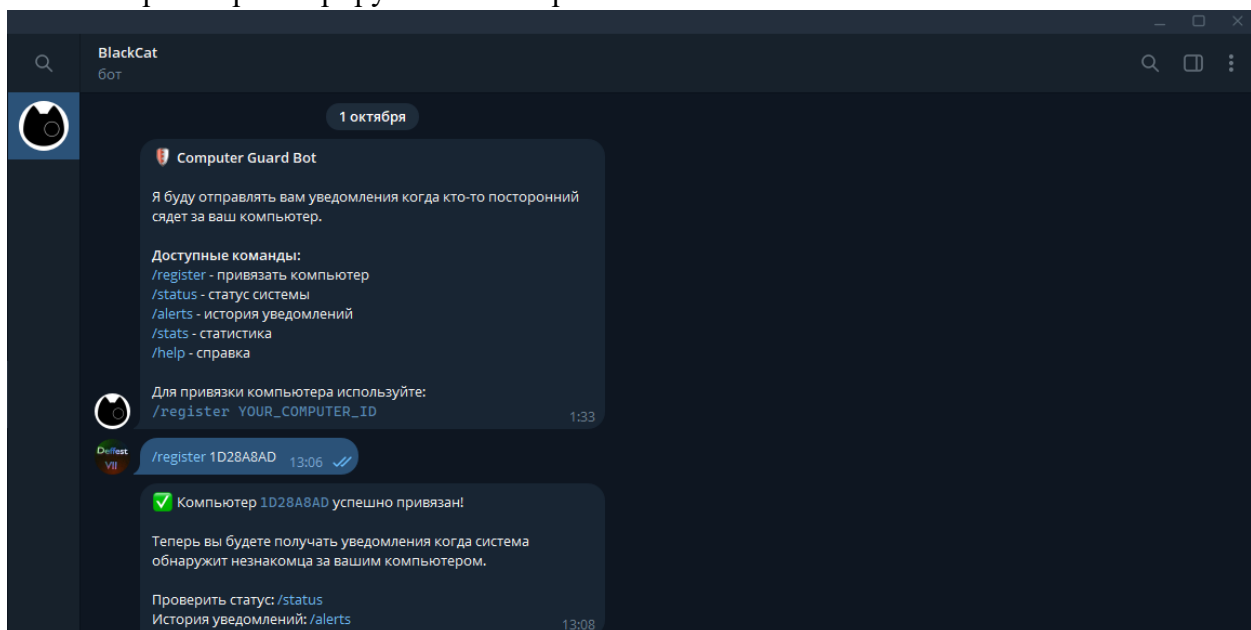
Мониторинг запущен! Система охраны активна.
Настройки обнаружения: 20 раз за 60 сек
Уведомление отправляется ОДИН РАЗ при достижении порога
Подойдите к камере для тестирования
Нажмите Ctrl+C для остановки

2025-10-01 14:49:50,160 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 148.9)
2025-10-01 14:49:50,160 - client.computer_guard - INFO - Обнаружен незнакомец...
2025-10-01 14:49:50,672 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 155.2)
2025-10-01 14:49:51,185 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 155.2)
2025-10-01 14:49:51,712 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 154.9)
2025-10-01 14:49:52,236 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 152.3)
Текущий счетчик: 5/20
2025-10-01 14:49:52,751 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 154.6)
2025-10-01 14:49:53,273 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 153.0)
2025-10-01 14:49:53,789 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 155.4)
2025-10-01 14:49:54,305 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 152.1)
2025-10-01 14:49:54,840 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 153.5)
Текущий счетчик: 10/20
2025-10-01 14:49:55,355 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 153.4)
2025-10-01 14:49:55,888 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 153.1)
2025-10-01 14:49:56,398 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 154.7)
2025-10-01 14:49:56,913 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 153.2)
2025-10-01 14:49:57,429 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 154.6)
Текущий счетчик: 15/20

```

3. Программа замечает меня и обозначает как незнакомца, так как модели не были обучены распознавать мое лицо, так как я еще не добавил свои фотографии.

4. Заметим, что также появился файл computer_config.json. Возьмем из него id нашего компьютера и зарегистрируемся в телеграмм боте:



5. Регистрация прошла успешно. Возвращаемся к start.bat и запускаем еще раз:

```

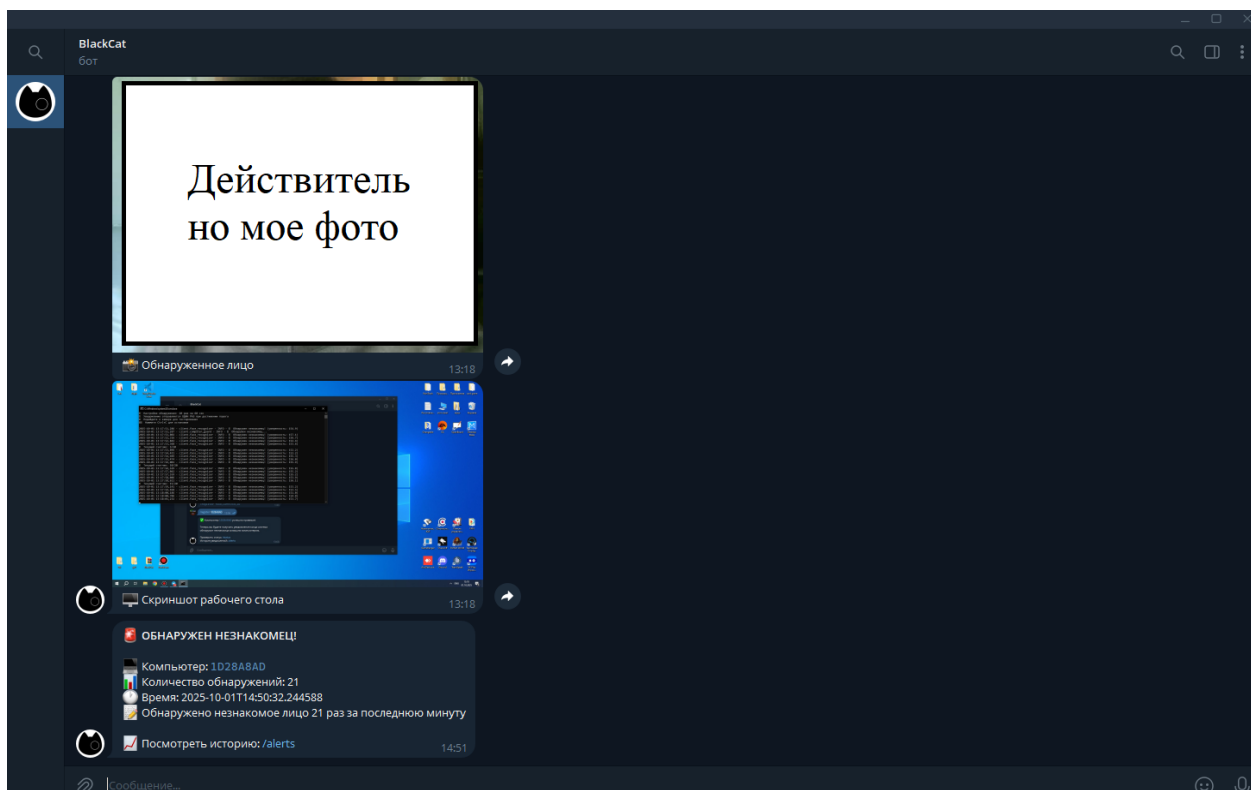
Запуск системы охраны...
Нажмите Ctrl+C для остановки
=====
2025-10-01 14:55:27,380 - client.computer_guard - INFO - Запуск мониторинга...
2025-10-01 14:55:28,538 - client.computer_guard - INFO - Камера успешно подключена

Мониторинг запущен! Система охраны активна.
Настройки обнаружения: 20 раз за 60 сек
Уведомление отправляется ОДИН РАЗ при достижении порога
Подойдите к камере для тестирования
Нажмите Ctrl+C для остановки

2025-10-01 14:55:31,157 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 164.1)
2025-10-01 14:55:31,157 - client.computer_guard - INFO - Обнаружен незнакомец...
2025-10-01 14:55:31,677 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 155.1)
2025-10-01 14:55:32,192 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 152.4)
2025-10-01 14:55:32,708 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 153.3)
2025-10-01 14:55:33,226 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 156.2)
Текущий счетчик: 5/20
2025-10-01 14:55:33,742 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 155.8)
2025-10-01 14:55:34,261 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 156.9)
2025-10-01 14:55:34,777 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 159.5)
2025-10-01 14:55:35,292 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 166.8)
2025-10-01 14:55:35,799 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 164.6)
Текущий счетчик: 10/20
2025-10-01 14:55:36,317 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 167.2)
2025-10-01 14:55:36,849 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 165.1)
2025-10-01 14:55:37,362 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 167.3)
2025-10-01 14:55:37,875 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 167.6)
2025-10-01 14:55:38,399 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 168.6)
Текущий счетчик: 15/20
2025-10-01 14:55:38,910 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 169.7)
2025-10-01 14:55:39,425 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 167.5)
2025-10-01 14:55:39,957 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 166.3)
2025-10-01 14:55:40,468 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 165.8)
2025-10-01 14:55:40,982 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 168.3)
2025-10-01 14:55:40,982 - client.computer_guard - INFO - Критическое количество обнаружений! Отправка уведомления...
Обнаружено 20 раз за минуту! Отправка уведомления...
2025-10-01 14:55:40,982 - client.computer_guard - INFO - Сохранено фото незнакомца: strangers_photos\stranger_1D28A8AD_20251001_145540.jpg
2025-10-01 14:55:41,067 - client.computer_guard - INFO - Сохранен скриншот: screenshots\screensh ot_1D28A8AD_20251001_145540.png
2025-10-01 14:55:41,083 - client.computer_guard - INFO - Отправка API запроса на: https://blackcat-mpix.onrender.com/api/alert
2025-10-01 14:55:43,145 - client.computer_guard - INFO - Уведомление успешно отправлено на сервер
2025-10-01 14:55:43,145 - client.computer_guard - INFO - Уведомление отправлено. Флаг установлен.
2025-10-01 14:55:43,670 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 165.6)
2025-10-01 14:55:44,188 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 163.1)
2025-10-01 14:55:44,700 - client.face_recognizer - INFO - Обнаружен незнакомец! (уверенность: 163.3)

```

6. Видим, что клиент уведомил нас о том, что он сделал нужные фотографии и отправил на сервер. Сервер в свою очередь сразу перенаправит фотографии в телеграмм бот, убедимся в этом, возвращаемся к нашему телеграмм боту:



Действительно видим, что телеграмм бот прислал нам фотографию нарушителя и фото рабочего стола.

Документация

1. OpenCV - Компьютерное зрение

- Документация: <https://docs.opencv.org/4.x/>
- Face Recognition: https://docs.opencv.org/4.x/da/d60/tutorial_face_main.html
- LBPH Recognizer: https://docs.opencv.org/4.x/dc/df6/tutorial_face_recognize.html

2. MediaPipe - Детекция лиц

- Документация: <https://google.github.io/mediapipe/>
- Face Detection: https://google.github.io/mediapipe/solutions/face_detection
- Python API: <https://google.github.io/mediapipe/api/solutions/python/>

3. FastAPI - Веб-сервер

- Документация: <https://fastapi.tiangolo.com/>
- Tutorial: <https://fastapi.tiangolo.com/tutorial/>
- Background Tasks: <https://fastapi.tiangolo.com/tutorial/background-tasks/>
- File Uploads: <https://fastapi.tiangolo.com/tutorial/request-files/>

4. Aiogram - Telegram бот

- Документация: <https://docs.aiogram.dev/en/latest/>
- Quick Start: https://docs.aiogram.dev/en/latest/tutorial/quick_start.html
- Webhook Guide: <https://docs.aiogram.dev/en/latest/tutorial/webhook.html>
- Handlers: <https://docs.aiogram.dev/en/latest/tutorial/handlers.html>

5. Supabase - База данных

- Документация: <https://supabase.com/docs>
- Python Client: <https://supabase.com/docs/reference/python>
- API Reference: <https://supabase.com/docs/reference/api>

6. PyAutoGUI - Скриншоты

- Документация: <https://pyautogui.readthedocs.io/en/latest/>
- Screenshot: <https://pyautogui.readthedocs.io/en/latest/screenshot.html>

7. Requests - HTTP запросы

- Документация: <https://docs.python-requests.org/en/latest/>
- Quick Start: <https://docs.python-requests.org/en/latest/user/quickstart/>
- File Upload: <https://docs.python-requests.org/en/latest/user/advanced/#post-multiple-multipart-encoded-files>

8. Pydantic - Валидация данных (часть FastAPI)

- Документация: <https://docs.pydantic.dev/latest/>
- Settings Management: https://docs.pydantic.dev/latest/concepts/pydantic_settings/

9. Python-dotenv - Переменные окружения

- Документация: <https://saurabh-kumar.com/python-dotenv/>

10. Uvicorn - ASGI сервер

- Документация: <https://www.uvicorn.org/>
- Deployment: <https://www.uvicorn.org/deployment/>

Литература

“Practical Security for Developers” - Roman Zabicki

“Programming Computer Vision with Python” - Jan Erik Solem

“Learning OpenCV 4 Computer Vision with Python 3” - Joseph Howse, Joe Minichino